

CloudShield AI / DAIL

12 — Testing & QA Specification

Version 1.0 | Final Testing/QA Baseline | 1 September 2026

Purpose: Define the complete quality strategy for DAIL before implementation. This specification establishes test layers, fixtures, deterministic unit tests, integration and system tests, adversarial cases, invariant/verifier tests, LLM boundary tests, evidence/audit tests, state-lifecycle tests, experiment correctness, CI quality gates, release criteria, defect handling, and the minimum Definition of Done for each implementation component.

Document Control

Field	Value
Document ID	CSAI-DAIL-QA-012
Status	FINAL TESTING/QA BASELINE
Parent documents	01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation; 09 Verification/Promotion; 10 LLM Integration; 11 Evidence/Logging
Primary outputs	Automated Test Suite, Fixtures, QA Reports, Release Gate
Quality principle	No safety-critical component is accepted without deterministic, adversarial, and end-to-end validation.
Test philosophy	Fail closed; prove behavior, not intent.

1. Scope

This document defines QA for the DAIL software system and its research/experiment harness. It covers application logic, infrastructure parsing/normalization, identity, dependency, impact, verification, promotion, LLM integration, evidence, storage, APIs, lifecycle, and operational failure handling.

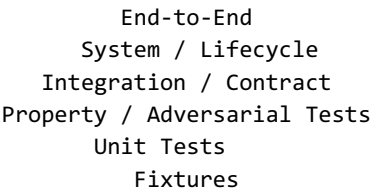
- Unit testing.
- Contract testing.
- Integration testing.
- System/end-to-end testing.
- Adversarial testing.
- Regression testing.
- Property/invariant testing.
- Fault injection.
- Reproducibility testing.
- CI/CD quality gates.
- Release acceptance.

2. Quality Principles

- Safety-critical paths fail closed.
- Tests assert observable semantics and persisted evidence.

- Every core algorithm has deterministic fixtures.
- Negative and adversarial cases are first-class.
- LLM output is always treated as untrusted.
- Tests must verify that invalid candidates cannot become Trusted State.
- Tests must verify that valid candidates cannot be rejected due to avoidable implementation defects.
- Historical evidence and state lineage must remain intact after failures.
- Research metrics must be generated from reproducible records rather than manual interpretation.

3. Test Pyramid



The majority of fast deterministic coverage should live at unit/property layers; a smaller number of high-value lifecycle tests validates the complete state machine.

4. Test Environment

Environment	Purpose
Local	Fast unit, parser, algorithm, and fixture tests.
CI	Repeatable automated test suite and quality gates.
Integration	Database/storage/provider-adapter integration.
Sandbox	Optional controlled AWS/Terraform validation.
Experiment	Reproducible research trials with immutable fixture/version metadata.

Production credentials and uncontrolled infrastructure are not required for the core safety test suite.

5. Test Data Strategy

- Use version-controlled Terraform fixtures.
- Use canonical normalized-state fixtures.
- Use expected IdentityReports/DependencyGraphs.
- Use expected ImpactReports and VerificationReports.
- Use expected PromotionDecision outcomes.
- Keep adversarial fixtures small and explainable.
- Never rely on mutable external cloud state for core unit tests.

6. Fixture Naming Convention

<domain>_<scenario>_<expected_outcome>_<version>

Examples:

sg_public_ssh_fail_v1
sg_private_ssh_pass_v1
ec2_rds_5432_pass_v1
ec2_rds_5432_break_fail_v1

Fixture IDs are stable references used in test results and experiment records.

7. Canonical Fixture Set

Fixture	Expected purpose
baseline_safe	Establish initial Trusted State.
public_ssh_violation	Security verifier FAIL.
public_ssh_repair	Security verifier PASS.
functional_reachable	EC2-RDS 5432 PASS.
functional_broken	EC2-RDS 5432 FAIL.
security_repair_breaks_function	Regression: SEC PASS, FUNC FAIL, promotion REJECT.
safe_repair	SEC PASS, FUNC PASS, promotion PROMOTE.
created_resource	Baseline verification required.
deleted_dependency	Dependent invariant affected.
replacement_resource	Replacement impact and evidence invalidation.
ambiguous_identity	UNCERTAIN correspondence.
unknown_dependency	Conservative impact behavior.
unsupported_relationship	UNSUPPORTED behavior.
stale_parent	Promotion blocked.
verifier_error	No promotion.

8. Unit Testing

Unit tests must cover every deterministic domain rule and pure transformation.

- Canonicalization/normalization.
- Resource fingerprints.
- Identity matching.
- Change classification.
- Reference extraction.
- Dependency edge derivation.
- Impact matching/traversal.
- Invariant predicates.
- Evidence validity.
- Decision aggregation.
- Promotion precondition checks.
- Hash generation.
- Redaction rules.

9. Identity Test Matrix

Test	Expected
Same address/type + same fingerprint	SAME + UNCHANGED.
Same address/type + changed semantics	SAME + MODIFIED.
Replacement evidence	REPLACED.
Trusted only	DELETED.
Candidate only	CREATED.
Two plausible matches	UNCERTAIN.

Weak heuristic only	UNCERTAIN.
Conflicting provider/address evidence	UNCERTAIN or explicit conflict outcome.
Unsupported identity semantics	UNSUPPORTED/UNCERTAIN.

10. Dependency Test Matrix

Test	Expected
Terraform reference	Typed edge with provenance.
Subnet→VPC	Known relationship.
Route-table association	Correct subnet/table edges.
EC2→SG	USES_SECURITY_GROUP.
RDS→SG	USES_SECURITY_GROUP.
SG rule→SG	RULE_BELONGS_TO_SECURITY_GROUP.
IGW relationship	Supported edge when configured.
Unknown reference	Unknown/unresolved, not silently absent.
Unsupported edge	UNSUPPORTED; no false negative.
Rule ordering change	Same semantic graph after canonicalization.

11. Impact Test Matrix

Scenario	Expected
Directly scoped change	Invariant AFFECTED.
Supported dependency path	Invariant AFFECTED.
Unrelated tag change	No impact unless explicitly scoped.
Created in-scope resource	Baseline verification work.
Deleted dependency	Dependent invariant affected.
Replacement	Conservative invalidation.
Identity uncertainty	Relevant evidence not safely carried forward.
Dependency uncertainty	No claim of unaffectedness.
Cycle	Traversal terminates.
Multiple paths	One affected record, multiple reasons.

12. Verification Test Matrix

Scenario	Expected
Security predicate satisfied	PASS.
Public SSH exists	FAIL.
Functional path established	PASS.
Functional requirement broken	FAIL.
Required context unavailable	UNKNOWN.
Unsupported required semantic	UNSUPPORTED.
Verifier exception	VERIFIER_ERROR.
Missing evidence metadata	Insufficient proof.
Wrong candidate hash	Result rejected as stale/incompatible.

13. Promotion Test Matrix

Condition	Expected
All required checks PASS	PROMOTE.
Any required FAIL	REJECT.
Required UNKNOWN	No promotion; RETRY/ESCALATE per policy.
Required UNSUPPORTED	No promotion.

Verifier ERROR	No promotion.
Stale parent	No promotion.
Evidence conflict	No automatic promotion.
Structural failure	REJECT.
DB commit failure	Previous Trusted State remains current.
Duplicate promotion request	Idempotent; no duplicate Trusted State.

14. State Lifecycle Tests

1. Create initial Trusted State.
2. Generate candidate from current parent.
3. Analyze candidate.
4. Verify affected invariants.
5. Reject unsafe candidate.
6. Confirm Trusted State remains unchanged.
7. Generate safe candidate.
8. Promote candidate atomically.
9. Confirm new Trusted State parent/version/evidence.
10. Attempt stale candidate promotion.
11. Confirm stale candidate cannot advance state.

15. Evidence & Logging Tests

- Every material decision has evidence references.
- Evidence hashes are stable.
- State/candidate bindings are correct.
- Invalidation preserves original evidence.
- Supersession is append-only.
- Audit events are correlated.
- Duplicate events are deduplicated.
- Secrets are redacted.
- Integrity failures block affected proof.
- Replay metadata is complete.

16. LLM Boundary Tests

- Valid structured response accepted.
- Malformed response rejected.
- Invalid HCL rejected.
- Unsupported resource rejected.
- Forbidden operation rejected.
- Prompt injection in Terraform comments cannot override instructions.
- Model confidence cannot promote candidate.
- LLM rationale cannot satisfy verification.
- LLM cannot modify promotion policy.
- LLM cannot mutate Trusted State.
- Context overflow cannot silently remove required constraints.
- Retry remains bounded.

17. Property-Based Testing

Where practical, use generated inputs to validate general properties rather than only hand-authored examples.

- Canonicalization is idempotent.
- Hash of canonical equivalent states is equal.
- Reordering unordered rules does not change semantics.
- Graph construction is deterministic.
- Adding an unrelated resource does not change unrelated invariant results.
- A required FAIL cannot aggregate to PASS.
- A required UNKNOWN cannot aggregate to PASS.
- Promotion never occurs without all required PASS evidence.
- Invalidated evidence cannot become valid without a new valid verification/carry-forward event.

18. Metamorphic Tests

Transformation	Expected property
Reorder security rules	Semantic result unchanged.
Reorder resource declarations where semantics unchanged	Identity/graph result unchanged.
Add irrelevant tag	Unrelated invariants unchanged.
Duplicate equivalent input representation	Canonical result unchanged.
Rename resource with changed Terraform address	Not automatically SAME.
Remove a required security rule	Security result cannot improve from FAIL to PASS incorrectly; expected predicate determines exact result.

19. Fault Injection

- Database unavailable during promotion.
- Evidence store unavailable.
- LLM timeout.
- LLM rate limit.
- Malformed LLM response.
- Verifier crash.
- Dependency parser failure.
- Hash mismatch.
- Stale candidate.
- Concurrent promotion attempt.
- Partial event delivery.

Every injected fault must produce a known, safe outcome and must not silently advance Trusted State.

20. Concurrency Tests

- Two candidates from same parent race to promote.
- Only one can become the next Trusted State under serialized/atomic policy.
- Second stale candidate is rejected/requeued.
- No duplicate state version is committed.
- Decision/evidence lineage remains internally consistent.

21. Security Testing

- Secret leakage in logs.
- Secret leakage in prompts.
- Prompt injection.
- Artifact path traversal.
- Unauthorized evidence mutation.
- Tampered evidence.
- Malformed Terraform input.
- Oversized request/context.
- Unsafe generated resource types.
- Attempt to bypass verifier/promotion boundary.

22. Parser Fuzzing

Terraform/HCL and structured LLM response parsers should receive malformed and boundary inputs.

- Truncated documents.
- Unexpected nesting.
- Duplicate fields.
- Very large strings.
- Unicode edge cases.
- Invalid references.
- Unexpected block types.
- Null/missing fields.
- Deeply nested structures.

Parser failures must be controlled errors, not process crashes or unsafe defaults.

23. Regression Suite

Every discovered production/fixture defect becomes a permanent regression test when the defect is representative of a supported behavior.

defect -> minimized fixture -> expected result -> automated regression test

- Record defect ID.
- Record component/version.
- Record expected safe behavior.
- Add fixture and assertion.
- Run in CI thereafter.

24. Golden Artifacts

For deterministic components, maintain golden expected outputs for selected fixtures.

- Normalized state golden files.
- Identity report goldens.

- Dependency graph goldens.
- Impact report goldens.
- Verification report goldens.
- Promotion decision goldens.

Golden artifacts must be versioned and updated only through intentional review.

25. Contract Testing

Boundary	Contract
Normalizer– Identity	Canonical resource schema.
Identity –Impact	IdentityReport schema.
Dependency– Impact	DependencyGraph schema.
Impact– Verification	VerificationWorkSet schema.
Verification –Promotion	VerificationReport schema.
Evidence– all components	Evidence/reference/hash contract.
LLM adapter– Candidate builder	LLMRepairResponse/Patch schema.

26. End-to-End Test Scenarios

26.1 Unsafe Repair

```

Baseline safe
↓
LLM proposes security repair
↓
Candidate removes public SSH
↓
Impact identifies security + functional work
↓
SEC = PASS
FUNC = FAIL
↓
PROMOTION = REJECT
↓
Trusted State unchanged

```

26.2 Safe Repair

```

Baseline unsafe
↓
LLM proposes repair
↓
Candidate validated
↓
SEC = PASS
FUNC = PASS
↓
PROMOTION = PROMOTE
↓
Trusted State vN+1

```


27. Test Oracle Strategy

For each safety-critical test, the expected result should be derived from a declared oracle or explicit fixture expectation.

Oracle	Use
Reference predicate	Expected invariant truth.
Independent implementation	Cross-check deterministic verifier.
Golden artifact	Exact deterministic output.
State-machine expectation	Lifecycle transition correctness.
Schema contract	Boundary correctness.

LLM output is never used as the oracle for its own safety.

28. Coverage Requirements

Coverage should be measured by behavior, not only line percentage.

- Every core decision branch has tests.
- Every failure class has at least one test.
- Every supported edge type has positive and negative tests.
- Every mandatory invariant has PASS/FAIL/UNKNOWN/UNSUPPORTED coverage.
- Every promotion decision outcome has coverage.
- Every state transition has positive and forbidden-transition coverage.

A numerical coverage threshold may be added by CI policy, but high line coverage cannot compensate for missing safety scenarios.

29. CI Quality Gates

Gate	Requirement
Formatting/lint	PASS.
Static analysis	PASS.
Unit tests	100% pass.
Contract tests	100% pass.
Core integration tests	100% pass.
Safety regression suite	100% pass.
Security/redaction tests	100% pass.
Build/package	PASS.
Migration/schema checks	PASS when applicable.
End-to-end smoke suite	PASS for release candidates.

30. Test Failure Policy

- Any safety-critical test failure blocks release.
- Flaky tests are not ignored; they are quarantined only with an explicit owner and tracking record.
- A quarantined test cannot cover a safety-critical requirement without an approved replacement.
- Failures must preserve logs/evidence for diagnosis.
- Release overrides require explicit recorded approval outside automated promotion logic.

31. Flaky Test Management

- Identify deterministic vs environmental flakiness.
- Record occurrence rate.
- Separate retry from pass semantics.
- Do not hide failures with unlimited retries.
- Fix root cause.
- Remove quarantine after stable evidence.

32. Performance Testing

Performance testing evaluates efficiency without weakening correctness.

- Identity analysis latency.
- Dependency graph construction latency.
- Impact traversal latency.
- Verification latency.
- Evidence persistence latency.
- End-to-end candidate cycle time.
- LLM generation latency/token usage where available.

Performance regressions are release concerns but cannot justify unsafe behavior changes.

33. Reproducibility Testing

- Same fixture + same algorithm versions → same deterministic output.
- Same experiment configuration records identical metadata.
- Golden hashes remain stable.
- Replay reconstructs the same logical decision.
- Stochastic LLM runs preserve exact configuration metadata even when outputs differ.

34. Experiment Harness QA

- Trial IDs are unique.
- Baseline state is immutable.
- Candidate attempts are linked to one parent.
- Model/prompt configuration is captured.
- Metrics are derived from persisted evidence.
- Failed trials remain analyzable.
- Comparisons use compatible fixture and policy versions.
- No manual edits to outcome data without audit trail.

35. Database/Storage Tests

- Schema constraints.
- Foreign-key/lineage integrity.
- Unique event IDs.
- Idempotent writes.

- Transaction rollback.
- Promotion atomicity.
- Hash persistence.
- Artifact reference resolution.
- Concurrent write handling.

36. API Tests

- Authentication/authorization boundaries.
- Schema validation.
- Invalid input rejection.
- Idempotency keys.
- Error status mapping.
- Correlation ID propagation.
- Candidate/state binding.
- Promotion endpoint stale-parent protection.

37. Release Candidate Checklist

- All automated CI gates pass.
- No open safety-critical defects.
- Core regression suite passes.
- Promotion transaction tested.
- Evidence integrity tested.
- Secret scanning/redaction passes.
- LLM boundary tests pass.
- Database migrations validated.
- End-to-end safe/unsafe scenarios pass.
- Version metadata is recorded.

38. Defect Severity

Severity	Meaning
S0	Potential unsafe promotion, trust-boundary bypass, or corruption of authoritative state.
S1	Safety-critical verifier/impact/lifecycle defect affecting required behavior.
S2	Major functional/observability defect without direct trust bypass.
S3	Minor defect, usability, documentation, or non-critical diagnostics.

S0/S1 defects block release until resolved or explicitly mitigated under controlled governance.

39. Definition of Test Complete

A feature is test-complete only when its requirements, normal behavior, failure behavior, boundary conditions, and integration effects are covered by automated tests appropriate to its risk.

- Unit tests exist.

- Negative tests exist.
- Integration contract exists.
- Failure path tested.
- Evidence/logging assertions exist.
- Regression fixture exists where applicable.
- Documentation references test IDs/results.
- CI gate is configured.

40. Definition of Done — Safety-Critical Component

Criterion	Required
Implementation complete	YES
Unit tests	PASS
Negative/adversarial tests	PASS
Integration tests	PASS
Evidence/audit tests	PASS
Regression tests	PASS
Static analysis/lint	PASS
Determinism tests	PASS where applicable
Failure injection	PASS where applicable
Code review	APPROVED
Documentation/spec alignment	VERIFIED

41. QA Traceability

Every major requirement should map to one or more test cases, and every safety-critical test should map back to a requirement/invariant.

Requirement → Component → Test Case → Fixture → Result → Evidence

42. Test Case Contract

```

TestCase {
  test_id
  requirement_ref[]
  fixture_ref
  setup
  action
  expected_result
  safety_class
  evidence_requirements[]
}

```

43. Test Run Contract

```

TestRun {
  run_id
  suite_version
  commit_sha
  environment
}

```

```

    started_at
    completed_at
    results[]
    artifact_refs[]
  }

```

44. QA Report Contract

```

QAReport {
  run_id
  release_candidate
  pass_count
  fail_count
  skipped_count
  safety_failures[]
  regression_failures[]
  coverage_summary
  environment
  decision = PASS | BLOCK
}

```

45. Acceptance Criteria

- All core deterministic algorithms have unit and adversarial coverage.
- All cross-component contracts have automated tests.
- Canonical safe and unsafe repair scenarios pass.
- Unsafe candidates cannot advance Trusted State.
- Safe candidates can advance when all requirements pass.
- UNKNOWN/UNSUPPORTED/ERROR outcomes never silently promote.
- Stale-parent protection is tested.
- Evidence integrity and lineage are tested.
- LLM outputs cannot bypass deterministic gates.
- CI blocks releases on safety-critical failures.
- Regression defects become permanent automated tests.
- Release candidates have reproducible QA reports.

46. Non-Goals

- Proving absence of all software defects.
- Testing every AWS service in the MVP.
- Relying exclusively on code coverage percentage.
- Using production infrastructure as the primary unit-test environment.
- Treating LLM output as a test oracle.

47. Final Testing & QA Directive

DAIL shall be developed under a fail-closed testing strategy in which safety-critical behavior is proven through deterministic fixtures, negative tests, adversarial cases, lifecycle tests, and end-to-end regression scenarios. A

passing build is not sufficient unless the trust boundary, evidence chain, invariant verification, and promotion transaction have all been tested.

The highest-priority QA property is invariant preservation: a candidate that improves one protected property while violating another must be demonstrably rejected, and a candidate may become Trusted only through the same evidence-backed verification and promotion path defined by Documents 06–11.

Appendix A — Mandatory MVP Test Families

- Baseline establishment.
- Public SSH violation.
- Public SSH repair.
- EC2–RDS 5432 reachable.
- EC2–RDS 5432 broken.
- Security repair breaks functional invariant.
- Safe repair preserves both invariants.
- Created resource baseline.
- Deleted dependency.
- Replacement.
- Ambiguous identity.
- Unknown dependency.
- Unsupported relationship.
- Stale parent.
- Verifier error.
- Evidence tampering.
- LLM malformed output.
- Prompt injection.
- Duplicate promotion.

Appendix B — Next Specification

Document 13 — Experiment Harness Specification defines how DAIL research experiments are executed, parameterized, seeded, measured, repeated, compared, and stored so that the effect of AI-generated repairs can be evaluated against deterministic and simpler baselines.